

Dokumentacja użytkownika – Airport Management System (REST API)

1) Opis aplikacji

Airport Management System to backendowa aplikacja (REST API) do zarządzania podstawowymi elementami działania lotniska:

- **Bramki (Gates)** – dodawanie, lista, dezaktywacja bramki
- **Loty (Flights)** – dodawanie, lista, zmiana statusu, przypisanie bramki
- **Pasażerowie (Passengers)** – dodawanie, lista, pobranie po ID, przypisanie lotu, check-in
- **Personel (Staff)** – dodawanie, lista, dezaktywacja pracownika
- **Grafik zmian (Shifts)** – dodawanie zmian, lista, lista zmian pracownika
- **Powiadomienia (Notifications)** – zapis powiadomień i symulacja wystania (log)
- **Raport KPI (Reports)** – szybki raport liczbowy (ile jest rekordów w modułach)

Aplikacja jest przeznaczona do obsługi przez narzędzia typu **Postman**, **curl** lub integrację z frontem.

2) Wymagania

- Java (JDK) zgodna z projektem (typowo 17+; u Ciebie lokalnie działało na nowszych)
- Maven
- MySQL (serwer bazy danych)
- (Opcjonalnie) MySQL Workbench do podglądu danych
- Postman do testowania API

3) Konfiguracja i uruchomienie

3.1 Konfiguracja bazy danych

Aplikacja korzysta z MySQL. Połączenie jest konfigurowane w:
src/main/resources/application.properties

Ważne ustawienia:

- spring.datasource.url=jdbc:mysql://localhost:3306/<nazwa_bazy>
- spring.datasource.username=...
- spring.datasource.password=...
- spring.jpa.hibernate.ddl-auto=update

Uwaga o trwałości danych:

Jeśli ddl-auto=update, dane **zostają** po restarcie aplikacji (dopóki nie usuniesz bazy).
Jeśli ustawisz create lub create-drop, dane będą kasowane przy starcie.

3.2 Uruchomienie aplikacji

Z katalogu projektu:

- mvn spring-boot:run

Domyślny adres:

- http://localhost:8081
-

4) Konta i uprawnienia (Basic Auth)

Aplikacja używa **Basic Authentication**.

Dostępne role:

- **ADMIN** – pełny dostęp (GET/POST/PATCH/DELETE)
- **STAFF** – dostęp tylko do odczytu (**GET** na /api/**)

Dane logowania:

- ADMIN: admin / admin123
- STAFF: staff / staff123

W Postmanie ustaw:

Authorization → Type: Basic Auth i wpisz login/hasło.

5) Zasady działania (co można, a czego nie)

5.1 Co można robić jako ADMIN

- dodawać i modyfikować dane (bramki, loty, pasażerów, personel, zmiany)
- przypisywać bramkę do lotu
- przypisywać lot pasażerowi
- wykonywać check-in pasażera
- wysyłać (symulowane) powiadomienia

5.2 Co można robić jako STAFF

- tylko przeglądać dane (listy, podgląd po ID, raport KPI)
- próba modyfikacji (POST/PATCH/DELETE) kończy się zwykle **403 Forbidden**

5.3 Ograniczenia biznesowe (przykłady)

- Pasażer **nie może** zrobić check-in, jeśli nie ma przypisanego lotu (zwróci błąd, np. 409).

- Duplikaty unikalnych wartości (np. kod bramki, numer paszportu) powodują konflikt (np. 409).

6) Endpointy i użycie (Postman)

6.1 Endpointy “systemowe”

GET {{baseUrl}}/health

GET {{baseUrl}}/

Nie wymagają autoryzacji.

7) Scenariusz użycia (najlepszy do demonstracji)

Poniższy flow pokazuje pełne możliwości aplikacji:

1. Dodaj bramki: A1, B2
2. Dodaj lot LO123
3. Przypisz bramkę do lotu
4. Dodaj pasażera
5. Przypisz pasażerowi lot
6. Zrób check-in
7. Dodaj pracownika
8. Dodaj zmianę (Shift)
9. Wyślij powiadomienie
10. Sprawdź raport KPI
11. Pokaż security: STAFF nie może robić POST

8) Przykładowe requesty (format Postman)

Zatóż: baseUrl = http://localhost:8081

8.1 Security demo (STAFF nie może modyfikować)

POST {{baseUrl}}/api/gates (Auth: STAFF)

Body (raw JSON):

```
{ "code": "A1", "active": true }
```

Oczekiwane: **403 Forbidden**

9) Moduły API

9.1 Gates

POST {{baseUrl}}/api/gates (ADMIN)

```
{ "code": "A1", "active": true }
```

GET {{baseUrl}}/api/gates (STAFF/ADMIN)

PATCH {{baseUrl}}/api/gates/{code}/deactivate (ADMIN)

Przykład: /api/gates/A1/deactivate

9.2 Flights

POST {{baseUrl}}/api/flights (ADMIN)

```
{  
  "flightNumber": "LO123",  
  "destination": "London",  
  "departureTime": "2026-01-11T14:30:00",  
  "status": "SCHEDULED"  
}
```

GET {{baseUrl}}/api/flights (STAFF/ADMIN)

PATCH {{baseUrl}}/api/flights/{flightNumber}/status/{status} (ADMIN)

Przykład: /api/flights/LO123/status/BOARDING

POST {{baseUrl}}/api/flights/{flightId}/gate/{gateId} (ADMIN)

Przykład: /api/flights/1/gate/1

flightId i gateId bierz z odpowiedzi JSON po utworzeniu.

9.3 Passengers

POST {{baseUrl}}/api/passengers (ADMIN)

```
{  
  "firstName": "Jan",  
  "lastName": "Kowalski",  
  "passportNumber": "P1234567"  
}
```

GET {{baseUrl}}/api/passengers (STAFF/ADMIN)

GET {{baseUrl}}/api/passengers/{id} (STAFF/ADMIN)

PATCH {{baseUrl}}/api/passengers/{id}/flight/{flightNumber} (ADMIN)

Przykład: /api/passengers/1/flight/LO123

PATCH {{baseUrl}}/api/passengers/{id}/check-in (ADMIN)

Przykład: /api/passengers/1/check-in

9.4 Staff

POST {{baseUrl}}/api/staff (ADMIN)

```
{  
  "firstName": "Anna",  
  "lastName": "Nowak",  
  "role": "CHECKIN_AGENT"  
}
```

GET {{baseUrl}}/api/staff (STAFF/ADMIN)

PATCH {{baseUrl}}/api/staff/{id}/deactivate (ADMIN)

Przykład: /api/staff/1/deactivate

9.5 Shifts

POST {{baseUrl}}/api/shifts (ADMIN)

```
{  
  "staffId": 1,  
  "day": "2026-01-12",  
  "timeFrom": "08:00:00",  
  "timeTo": "16:00:00"  
}
```

GET {{baseUrl}}/api/shifts (STAFF/ADMIN)

GET {{baseUrl}}/api/shifts/staff/{staffId} (STAFF/ADMIN)

Przykład: /api/shifts/staff/1

9.6 Notifications

POST {{baseUrl}}/api/notifications (ADMIN)

```
{  
  "type": "INFO",
```

```
"recipient": "all",  
"message": "Boarding for LO123 starts now."  
}
```

GET {{baseUrl}}/api/notifications (STAFF/ADMIN)

Uwaga: wysłanie jest symulowane – w logach aplikacji pojawia się informacja o “wysyłce”.

9.7 Reports (KPI)

GET {{baseUrl}}/api/reports/kpi (STAFF/ADMIN)

Zwraca szybkie statystyki (np. liczba lotów, pasażerów, zmian).

10) Jak wyczyścić środowisko przed prezentacją (czysta baza)

Najprościej (MySQL):

```
DROP DATABASE IF EXISTS airportdb;
```

```
CREATE DATABASE airportdb;
```

Potem uruchom aplikację ponownie – tabele utworzą się automatycznie (przy ddl-auto=update).

Alternatywa bez kasowania: zmień nazwę bazy w spring.datasource.url na np. airportdb_demo.

11) Najczęstsze problemy i szybkie rozwiązania

1) 401 Unauthorized

- brak Basic Auth w Postmanie albo złe dane logowania.

2) 403 Forbidden

- próbujesz zrobić POST/PATCH/DELETE jako STAFF.

3) 500 / problem z bazą

- MySQL nie działa, złą nazwą bazy, złe hasło, brak uprawnień.
- sprawdź application.properties i czy MySQL jest uruchomiony.

4) Konflikt duplikatu (409)

- próbujesz dodać obiekt z unikalnym polem, które już istnieje (np. gate code, passport).